

100% de cobertura, nove bugs e a suíte toda verde: *como um número certo esconde um software errado*

DOUGLAS QUEIROZ · INSTRUTOR · QA / AUTOMAÇÕES

• A pergunta que abre a aula

“*A cobertura do nosso projeto está em 100%. Então está tudo testado, né?*”

Essa frase aparece em toda equipe que liga o relatório de cobertura pela primeira vez, e ela é honesta: se o número é 100, o que exatamente estaria faltando? Responder “o syllabus diz que não” não convence ninguém — e nem deveria. Argumento de autoridade não muda a cabeça de quem está olhando um relatório todo verde.

Então esta aula não pede que você acredite. Ela pede que você **rode o código e veja**. Existe um projetinho preparado com um objetivo bem específico: o relatório precisa cravar 100% em tudo, e precisa haver defeito real em cada arquivo lá dentro.

São **nove arquivos JavaScript com um defeito clássico em cada um** e uma suíte de 20 testes que passa inteira, com **100% de comandos, 100% de ramos, 100% de funções e 100% de linhas**. Defeitos encontrados por ela: **nenhum**. É o experimento inteiro em uma frase.

• O caso mais curto de explicar: o bug mora na fronteira

Uma função classifica a severidade de um incidente pelo tempo que o sistema ficou fora do ar. A regra combinada com o time: a partir de 240 minutos é crítica, a partir de 60 é alta, acima de zero é média, zero é baixa.

```
// o que foi escrito – repare no > onde a regra pedia >=
function classificarSeveridade(minutosForaDoAr) {
  if (minutosForaDoAr > 240) return 'critica';
  if (minutosForaDoAr > 60)  return 'alta';
  if (minutosForaDoAr > 0)   return 'media';
  return 'baixa';
}

// o que foi testado – 300, 120, 10 e 0
expect(classificarSeveridade(300)).toBe('critica'); // ✓
expect(classificarSeveridade(120)).toBe('alta');   // ✓
expect(classificarSeveridade(10)).toBe('media');    // ✓
expect(classificarSeveridade(0)).toBe('baixa');     // ✓ → 100% de linha e de ramo
```

Quatro testes verdes, todos os ramos exercitados, relatório impecável. E o incidente de **exatamente 240 minutos** sai classificado como “alta” em vez de “crítica”. O defeito está na borda; os testes passaram no meio do intervalo e nunca encostaram nela.

Liga com a técnica: é exatamente isto que a **Análise de Valor Limite** (CTFL 4.0, seção 4.2.2) manda fazer — testar 239, 240 e 241, não 300. A técnica de caixa-preta encontra em três casos o que a cobertura de caixa-branca não viu em quatro.

Os outros oito casos seguem a mesma lógica: tipo inesperado que vira resultado errado em silêncio, `||` no lugar de `&&` com cobertura de ramo completa, teste com `toBeDefined()` que executa a linha e não verifica nada, função que muta o array de quem chamou, `await` esquecido no teste assíncrono, laço rodado uma vez só, aritmética de dinheiro em float — e o mais caro de todos, o nono.

• O furo que ferramenta nenhuma encontra

O nono caso é uma regra de negócio que **simplesmente não foi implementada**. O requisito dizia “só aceitar inscrição se houver vaga na turma”. A verificação nunca virou código. O teste unitário está verde, a cobertura está em 100% — e a turma estoura o limite no dia da aula.

Requisito esquecido não gera linha. Linha que não existe não pode ficar descoberta. O relatório continua em 100%. Cobertura mede a fração do código que *existe* — nunca a fração do requisito que virou código.

Guarde este como o ponto cego estrutural da métrica. Quem o fecha não é ferramenta: é **revisão de requisitos e teste estático** (capítulo 3 do syllabus) e **rastreabilidade da base de teste até o caso de teste** (seção 1.4.4). Por isso a aula insiste que cobertura precisa ser medida contra os requisitos, não só contra o código.

• Por que 100% não é 100%

- **Cobertura mede execução, não verificação.** O contador sobe quando a linha roda. Ele não sabe se você olhou o resultado. Um teste sem asserção — ou com asserção fraca — cobre tudo e prova nada.
- **Existem vários critérios, e o mais fácil é o mais fraco.** Comando é fraco; ramo é médio; condição / MC-DC é forte; caminho é inviável na prática. Quando alguém diz “100% de cobertura”, quase sempre está falando do primeiro da lista.
- **Cobertura não mede dados.** A linha `total += valor` fica coberta com `10`. Continua coberta — e continua errada — com `0.1`, `-5`, `NaN` e `undefined`.
- **Cobertura não mede tempo, ordem nem estado.** Concorrência, `await` esquecido, cache sujo, variável de módulo que sobrevive entre chamadas. A linha roda; o problema está em *quando* e *quantas vezes*.
- **Cobertura não mede integração nem o mundo real.** 100% de cobertura unitária não diz nada sobre o banco fora do ar, a API que mudou de contrato, o fuso do servidor, a rede lenta ou a tela com dez mil registros. É por isso que a pirâmide de testes tem mais de um andar.

- **O juiz de verdade: teste de mutação**

Se cobertura não mede a força da suíte, o que mede? A diferença cabe em duas perguntas:

Cobertura pergunta: “o teste passou por essa linha?”

Mutação pergunta: “se eu quebrar essa linha de propósito, algum teste reclama?”

A ferramenta copia o projeto, altera o código sozinha — troca `+` por `-`, `>=` por `>`, `&&` por `||`, esvazia o corpo de uma função, apaga o texto de uma mensagem de erro — e roda a suíte contra cada versão alterada. Cada versão é um **mutante**. Se algum teste falha, o mutante **morre**: a suíte estava vigiando aquilo. Se todos passam, ele **sobrevive** — e sobreviveu porque ninguém verifica aquele comportamento. O score é a proporção de mutantes mortos sobre os válidos.

O experimento roda nas duas pontas. Mesmo tipo de código, suítes diferentes:

MÉTRICA	SUÍTE FRACA (A “ILUSÃO”)	SUÍTE FORTE (DEPOIS DO CONSERTO)
Testes	20	122
Comandos / ramos / funções / linhas	100%	100%
Score de mutação	79,59% 20 de 98 sobreviveram	98,57% 3 de 210 sobreviveram
Defeitos encontrados	nenhum	todos

As duas colunas têm exatamente a mesma cobertura. **A diferença só aparece na linha da mutação** — e é aí que está a aula inteira. O relatório de sobreviventes é uma lista nominal de buracos: arquivo, linha e a alteração exata que ninguém percebeu.

E vem junto a honestidade sobre o limite da técnica: alguns mutantes **não podem ser mortos**, porque a alteração não muda o comportamento observável. São os **mutantes equivalentes** — os três que sobraram — e detectá-los automaticamente é um problema indecidível. Ou seja: 100% de score de mutação normalmente é inatingível, e perseguir esse número é desperdício. Ele serve para você abrir o relatório e olhar cada sobrevivente, decidindo caso a caso se é buraco de teste, código redundante ou mutante equivalente.

Honestidade sobre o escopo: teste de mutação **não é conteúdo de prova da CTFL**

Foundation — ele aparece no nível Advanced. Está aqui porque é a forma mais direta de demonstrar, na prática, o que o syllabus afirma no primeiro princípio do teste — e porque o mercado usa.

- **O que a mutação mandou consertar no código**

A parte que costuma surpreender a turma: dois sobreviventes apontaram **código redundante**, não teste faltando. O `typeof v === 'number'` antes de `Number.isFinite(v)` nunca fez diferença — `Number.isFinite` já devolve `false` para `'5'`, `true`, `null`, `[]` e `{}`. E o `if (desconto > 0)` era um atalho inútil, porque com desconto zero a própria conta já devolvia o valor certo. Ramo a menos é teste a menos para manter.

O checklist que sai daí, válido para qualquer função em qualquer projeto:

1. **Guarda de contrato no topo.** Valide tipo e faixa antes de calcular e falhe alto — `throw new TypeError(...)`, não `return undefined`. Entrada errada sem guarda vira resultado errado em silêncio, e silêncio não aparece em relatório nenhum.
2. **Uma condição por motivo, sem redundância.** Condição redundante é ramo a mais para cobrir e mutante que sobrevive sem apontar bug nenhum.
3. **Mensagem de erro específica e testável.** Ela permite a asserção `.toThrow('...')` — que é o que mata mutante de literal string.
4. **Função pura por padrão.** Nunca mute o argumento recebido (`[...lista].sort()`, não `lista.sort()`) nem acumule em variável de módulo.
5. **Zero, um e muitos são três casos distintos.** Sempre. Trate a coleção vazia explicitamente, antes de qualquer divisão.
6. **Dinheiro nunca em float.** Converta para centavos e normalize o ruído — e documente no teste se o arredondamento é por item ou só no total.
7. **Assíncrono sempre com `await` no teste.** Sem ele, o caso termina antes da rejeição acontecer e o teste passa verde por engano.

Um detalhe que virou a frase preferida do projeto: a primeira versão “corrigida” da função de dinheiro já tinha 100% de cobertura e continuava errada em `1.005`, porque o erro do float não some ao multiplicar por 100 — só muda de casa. **Corrigido é sempre “corrigido até o próximo teste”.**

• Como escrever o teste que mata o mutante

MUTANTE	COMO MATAR
<code>x >= 240</code> → <code>x > 240</code>	Testar exatamente 239, 240 e 241
<code>a && b</code> → <code>a b</code>	Testar cada operando isolado (tabela-verdade)
<code>if (cond)</code> → <code>if (true)</code>	Testar o caminho em que <code>cond</code> é falsa e o resultado muda
<code>'mensagem'</code> → <code>''</code>	Asserção com <code>.toThrow('mensagem exata')</code>
<code>a + b</code> → <code>a - b</code>	Usar valores em que soma e subtração diferem (evitar <code>0 + 0</code>)
corpo da função → <code>{}</code>	Ter pelo menos uma asserção sobre o valor de retorno

• O efeito colateral de transformar métrica em meta

O projeto tem `coverageThreshold` configurado em 100 para os quatro critérios — exatamente o portão de qualidade que muitas equipes colocam no CI achando que ele garante software sem defeito. **Ele passa no portão com nove arquivos defeituosos dentro.**

E existe a armadilha, que tem nome: **Lei de Goodhart** — *quando uma medida vira meta, ela deixa de ser uma boa medida*. Coloque “100% de cobertura” como obrigação e observe o que aparece em três sprints: teste que chama a função e não verifica nada; comentário de *ignore* espalhado pelo código; teste escrito depois, copiando o valor esperado da saída do próprio código (ou seja, documentando o bug como se fosse a regra); `if` removido “para melhorar a cobertura”. Nada disso é má-fé — é o time entregando exatamente o que a métrica pediu.

Por isso, aqui, cobertura entra como **indicador de apoio**, nunca como KPI isolado. O que dá para acompanhar junto: densidade de defeitos por módulo e defeitos escapados para

produção; score de mutação nos módulos críticos; rastreabilidade de requisito até caso de teste (é o que pega o nono caso); tempo médio para detectar e corrigir. E vale para a mutação também: se o score virar meta, o time gasta semanas perseguindo mutante equivalente — que, por definição, não pode ser morto.

• Então cobertura não serve para nada?

Serve — e muito. Só não serve para o que costumam usar. **Cobertura é ótima lida ao contrário:** o número alto não prova nada; o número baixo **denuncia**. Abra o relatório e olhe as linhas vermelhas: ali estão os tratamentos de erro que ninguém exercita, o `catch` que nunca rodou, a regra de negócio que entrou sem teste. É uma **lista de perguntas**, não um selo de qualidade.

Cobertura baixa é evidência forte de que falta teste.

Cobertura alta não é evidência de que o teste é bom.

• Onde isso encosta no syllabus

O QUE VOCÊ VIU NA PRÁTICA	ONDE ESTÁ NA CTFL 4.0
Nove bugs com a suíte verde	1.3 — Princípios: “testes mostram a presença de defeitos, não a sua ausência” e “testes exaustivos são impossíveis”
O requisito que não virou código	1.4.4 — Rastreabilidade; capítulo 3 — Teste estático
Testar 239, 240 e 241	4.2.2 — Análise de valor limite
Zero, um e muitos; válido e inválido	4.2.1 — Particionamento de equivalência
A tabela-verdade do <code>&&</code>	4.2.3 — Teste por tabela de decisão
Comandos, ramos e o que eles não garantem	4.3.1 , 4.3.2 e 4.3.3 — Técnicas de caixa-branca
Cobertura no portão de qualidade	5.1.5 — Critérios de saída; 5.3 — Monitoramento e controle

• Leve daqui

- Cobertura é um **mapa de onde o teste passou**, não um atestado de que o software funciona.
- Todo relatório tem um ponto cego do tamanho do requisito que ninguém implementou.
- Em JavaScript, tipo errado quase nunca vira exceção — vira resultado errado em silêncio. Teste `null`, `undefined`, string, array, negativo, zero e valor gigante.
- Teste o valor limite, não o meio do intervalo. O bug mora na fronteira.
- Se o seu teste não tem uma asserção específica, ele não é um teste: é um aquecimento.
- Métrica boa é métrica que você usa para **perguntar**, não para **comemorar**.

- **Nota final: como este material foi montado**

Coerência exige confissão: **o projeto e este texto foram construídos com apoio de IA** — e precisaram exatamente do mesmo tratamento que a gente discutiu na aula de IA.

Nove arquivos com defeito, três suítes, 142 testes, duas configurações de mutação e a documentação saíram em uma tarde. Na mão seriam semanas, e eu provavelmente teria desistido no terceiro caso. Só que a IA não sabia *quais* nove defeitos valiam ser ensinados — essa lista veio de anos vendo os mesmos erros voltarem em revisão de código. Ela também não sabia que o caso do requisito ausente precisava ser o último e o mais enfático, porque é o único que nenhuma ferramenta encontra. E a primeira versão do texto vendia teste de mutação como bala de prata; corriji para incluir o mutante equivalente e o custo real de rodar isso todo dia. **A IA executa numa velocidade que a gente não alcança. A direção continua sendo nossa.**

Quer rodar? O projeto da aula está no repositório da turma: um `npm install` e quatro comandos — a suíte que ilude, a que expõe os bugs, a que blinda o código e a mutação que julga as duas. Roda offline, sem banco e sem navegador, e gera relatórios navegáveis com o diff de cada mutante sobrevivente. O roteiro de sala e os desafios estão em `docs/exercicios.md`.

“Cobertura responde que parte do código o seu teste visitou. Ela nunca respondeu se o software está correto.”